Agent Native 工作方法论

- Agent 的能力已经够强了，上下文不足才是任务失败的主因。
- 从 Day 0 就用 agent 工作，让上下文随项目自然积累，而非事后补齐。
- Agent = 上下文体（git 仓库）+ 运行时（任意 coding agent），框架无关、模型无关。
- 交接项目就是交出上下文体，协作成本趋近于零。
- Daily notes 记录过程，doc 沉淀知识，SOP 从实践中提炼可复用流程。

为什么需要 Agent Native

Agent 的工作能力已经够强了。很多时候 agent 完不成任务，不是能力问题，是上下文不够。

大模型的权重只包含通用的专家知识。公司内部的最新项目、客户的态度、开发项目的前因后果、设计上的细微考量，这些从未出现在训练集中的信息，模型没有理由知道。

而把这些上下文传递给 agent 很难。人类只能用文字缓慢输出，带宽就那么大。更麻烦的是，人类往往意识不到自己有多少隐含知识没有传达。就算意识到了，愿意写，持续同步的代价也很大。

所以一个项目跑了一段时间再引入 agent，agent 会面临巨大的上下文缺口，效果很难好。但如果从 Day 0 就用 agent，让 agent 每次工作时自己记录前因后果，上下文就会随项目自然积累。这相当于把文档沉淀的成本均摊到了每次 session 中，每次多记一点，比事后补齐轻松得多。

这是 Agent Native 的第一层意思：从一开始就按 agent 的方式工作，让上下文随项目生长。

用 agent 做协作

Agent 同时也是协作的载体。

传统的工作交接需要写文档、开会、讲前因后果，本质上都是在传递 context，效率很低。用 Agent Native 的方式，交接就是把上下文体交出去。接手的人可以跑一个 coding agent 在这个上下文体上提问，也可以直接翻文件。大部分信息已经在 agent 的记录里了，不需要额外的同步。

Agent 的定义

在这套方法论里，agent 分两部分：

上下文体（静态）是一个 git 仓库，包含项目代码、daily notes、doc、SOP 等全部文件。它可以交接、可以持久化，承载了 agent 跨 session 的连续性。

运行时（动态）是任意 coding agent 程序加上背后的大模型（比如 Claude Code）。运行时在上下文体之上启动，读取信息，执行任务，把成果写回去。

上下文体是通用的，运行时可以换。所以这套方法论不绑定任何 agent 框架或基础模型。

核心组件

Daily notes

Agent 的工作日志。每次 session 的发现、决策、进展都记在里面，是上下文积累的主要手段。

- Append-only，不改历史记录
- 要更正就显式标注，不要静默修改

Doc

项目的知识库，放在 `doc/` 目录下。来源不限：agent 的总结、人写的文档、外部参考资料都行。Doc 是对项目某个方面的认知快照。

SOP

标准操作流程，从 daily notes 的实践中提炼，放在 `doc/sop/` 目录下。SOP 由 agent 执行而不是脚本执行，因为 agent 能处理过程中的意外。

每个 SOP 包含：起始条件、结束条件、执行步骤。

SOP 的创建由人决定，agent 可以建议。

工作流程

Agent 由 prompt 启动，prompt 来自用户或自动化系统，内容是具体的工作任务。Agent 不需要启动时读取所有 daily notes 和 doc，而是根据当前任务按需查阅，自己判断该读什么。

信息沉淀的路径：daily notes 积累实践经验，再提炼为 SOP。Doc 独立积累，来源不限。

使用技巧

随时写 daily notes，不要依赖 compact。很多 coding agent 有 compact（压缩上下文）功能，但 compact 要为压缩预留一部分上下文空间。更好的做法是：觉得有东西值得记就立刻写进 daily notes 文件。这样既持久化了信息，又给当前任务腾出了上下文空间，不用等 context 快满了再 compact。

案例：昇腾 910B 集群运维

昇腾 910B 集群上的 DeepSeek-R1 部署与运维项目从 Day 0 就用 agent 驱动。一个月下来积累了：

- 87 篇 daily notes，覆盖驱动升级、网络调试、性能压测、故障排查的全过程
- 20 多个 SOP，按阶段组织为环境构建、节点基础设施、网络诊断、部署操作、故障排查五个阶段，有完整的依赖关系图
- 多份 doc，包括内部性能分析、客户版报告、可行性研究，agent 写的和人写的都有

这里面能看出几件事。87 篇 daily notes 不是专门写的文档，就是每次 session 的工作记录，新的 session 按需翻阅就能拿到历史上下文。SOP 也不是凭空设计的，比如"HCCL Timeout 排查"这个 SOP 就是撞了好几次同样的问题之后总结出来的。整个上下文体通过 git 同步，新的协作者直接对接就行，不用从零了解项目。

常用 prompt 示例

以下是实际使用中常见的几种 prompt，供参考。

开始工作：让 agent 接续之前的任务

> 看一下 daily-notes 里最近的记录，继续完成 XXX 的工作。

Agent 会自己去翻 daily notes，找到上次的进度，然后接着干。

工作结束：让 agent 收尾

> 把这次的工作总结记录到 daily notes，然后 git commit push。

一次 session 结束时说一句就行。agent 负责总结写入和提交。

中途发现重要信息：立刻记录

> 这个发现比较重要，马上记到 daily notes 里。

不用等到 session 结束。重要的东西随时写，一方面防丢，一方面释放上下文空间。

让 agent 提炼 SOP

> 这个流程我们已经做过好几次了，帮我整理成一个 SOP 放到 doc/sop/ 下。

Agent 会回溯 daily notes 里相关的记录，提炼出标准流程。

交接给新协作者

> 我是新接手这个项目的，帮我了解一下整体情况。

新的人跑一个 agent session，agent 会去读 doc 和 daily notes 来回答。

开放问题：多 agent 协作

有些事情单个 agent 做不了，那就需要多个 agent。多个 agent 之间最大的区别是它们的上下文体不同。

上下文体怎么变得不同的？两种情况：一是两个本来就独立的项目，各自有各自的上下文体，到某天需要协作了；二是一个项目的上下文体越长越大，大到单个 agent 吃不下了，只好拆开。

拆分的判断标准、多 agent 之间怎么协作，目前还没有好的答案。